Fatima Jinnah Women University

Department of Software Engineering

Assignment 01

Git, Gitea, GitHub, Git LFS, and GitHub Pages

Student Name	**Shanza Anam**
Registration No.	**FA24B1-SE-049**
Course	**Cloud Computing (BSE-410)**
Program / Semester	**BS Software Engineering (BS(SE)-V)**
GitHub Username	**shanzaanum7-hub**
Submission Date	**10 October 2026**
Total Marks	**10**

Table of Contents

No.	Section
1	Introduction and Objectives
2	Task 1: Install Gitea and Push a Repository
3	Task 2: Push the Same Repository to GitHub
4	Task 3: Track Three Large Files with Git LFS
5	Task 4: Portfolio with GitHub Pages
6	Summary of Deliverables
7	Conclusion

1. Introduction and Objectives

This report documents the practical work completed for Assignment 01 of Cloud Computing. Gitea, a self-hosted Git service, was installed and run in Docker on an Ubuntu server. A local Git repository was created on the same server and pushed to both Gitea and GitHub, three large files were managed with Git LFS, and a portfolio website was published using GitHub Pages. Every step was carried out by me, and each required screenshot was captured immediately after the corresponding step.

Learning outcomes achieved

- Installed, ran and accessed a Gitea service on an Ubuntu server.
- Created and managed a Git repository from the Ubuntu server.
- Worked with multiple Git remotes (Gitea and GitHub).
- Authenticated safely with personal access tokens without exposing credentials.
- Tracked and pushed large files using Git LFS.
- Published a static portfolio with GitHub Pages.

Working environment

Item	Details
Server OS	Ubuntu 26.04.1 LTS (hostname: ubuntu)
Server IP address	192.168.215.130
Terminal prompt	shanzaanum7-hub@ubuntu:~$
Tools used	Git 2.53.0, Docker 29.1.3, Docker Compose v5.6.0, Git LFS 3.7.1
Gitea	Version 28.1.0 with PostgreSQL 17 (Docker containers), port 3000
Access method	SSH from Windows PowerShell; Chrome browser for Gitea and GitHub

2. Task 1: Install Gitea and Push a Repository

Step 1.1: Install and run Gitea on the Ubuntu server

I connected to the Ubuntu server over SSH, installed Docker Engine and curl, and downloaded the Docker Compose plug-in. After adding my account to the docker group I signed out and back in, then verified the Git, Docker and Docker Compose versions. I then cloned the instructor-provided Gitea setup repository and started the containers with the supplied compose.yaml, which I did not modify.

```
sudo apt update -y
sudo apt install -y docker.io curl
sudo mkdir -p /usr/local/lib/docker/cli-plugins
sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64
\
  -o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose
sudo systemctl start docker
sudo usermod -aG docker $USER
git --version && docker --version && docker compose version
git clone https://github.com/WaqasSaleem97/Gitea.git
cd Gitea && docker compose up -d
docker compose ps
curl -sS -o /dev/null -w "Gitea HTTP status: %{http_code}\n" http://127.0.0.1:3000/
```

In the browser I opened http://192.168.215.130:3000/ and completed the Gitea installation page with the required settings:

Setting	Value used
Database type	PostgreSQL
Database host	db:5432
Database username / password / name	gitea / gitea / gitea
Gitea base URL	http://192.168.215.130:3000/
Administrator account	shanzaanum7-hub

Result: both containers, gitea and gitea_db, are Up, and Gitea answers on port 3000 with HTTP status 200.



Figure 1: Gitea and PostgreSQL containers running, HTTP status 200 *(task1_gitea_running.png)*

Step 1.2: Create and push the initial repository

An empty public repository named Assignment01 was created in Gitea. On the server, a separate local repository was created in the home directory (outside both the Gitea folder and the CC repository). A README.md with my name and registration number was committed. A Gitea personal access token (name: ubuntu-server-token, repository: Read and Write) was generated. The gitea remote was added with only my username in the URL, and the token was entered only at the password prompt so it never appears in any command, URL or screenshot.

```
cd ~ && mkdir Assignment01 && cd Assignment01
git init
git branch -M main
git add README.md
git commit -m "Add student information"
git remote add gitea http://shanzaanum7-hub@127.0.0.1:3000/shanzaanum7-hub/Assignment01.git
git remote -v
git push -u gitea main
```

Result: the push succeeded (* [new branch] main -> main) and the branch now tracks gitea/main. The remote URL contains no password or token.

```
shanzaanum7-hub@ubuntu:~/Assignment01$ git remote -v
gitea   http://shanzaanum7-hub@127.0.0.1:3000/shanzaanum7-hub/Assignment01.git (fetch)
gitea   http://shanzaanum7-hub@127.0.0.1:3000/shanzaanum7-hub/Assignment01.git (push)
shanzaanum7-hub@ubuntu:~/Assignment01$ git push -u gitea main
Password for 'http://shanzaanum7-hub@127.0.0.1:3000':
shanzaanum7-hub@ubuntu:~/Assignment01$ git push -u gitea main
Password for 'http://shanzaanum7-hub@127.0.0.1:3000':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 275 bytes | 68.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To http://127.0.0.1:3000/shanzaanum7-hub/Assignment01.git
 * [new branch]      main -> main
branch 'main' set up to track 'gitea/main'.
shanzaanum7-hub@ubuntu:~/Assignment01$
```

Figure 2: git remote -v and successful push to the gitea remote (task1_gitea_push.png)

Step 1.3: Verify the Gitea repository

The repository page in Gitea shows the owner shanzaanum7-hub, the name Assignment01, the commit "Add student information", and the rendered README with my full name and registration number. The Gitea URL is visible in the address bar.

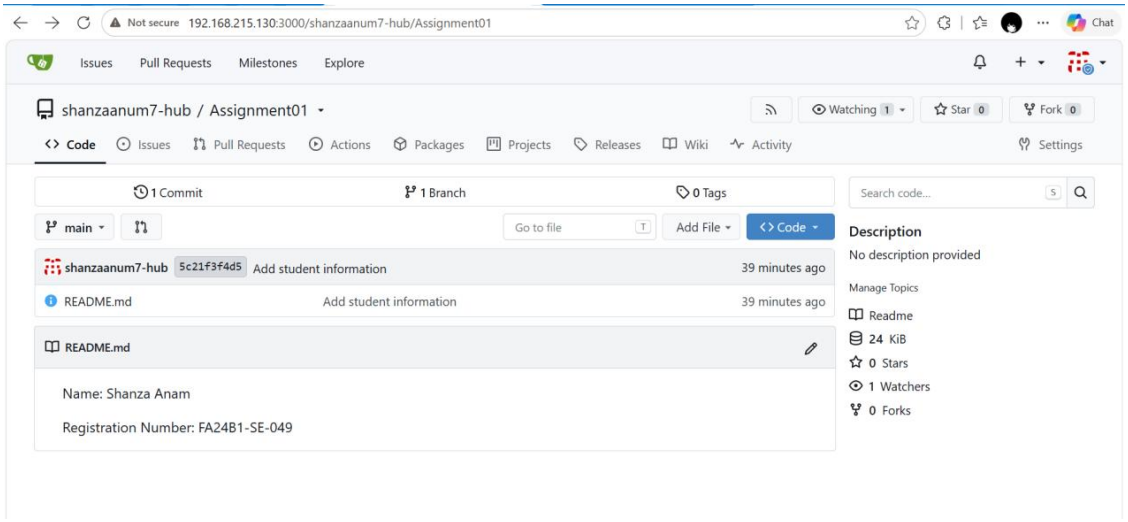


Figure 3: Gitea repository page with rendered README (task1_gitea_repository.png)

Task 1 result: Gitea was installed and run on the Ubuntu server, a separate local repository was created there, and it was pushed to Gitea over the local loopback interface.

3. Task 2: Push the Same Repository to GitHub

Step 2.1: Add GitHub as a second remote

An empty public repository named Assignment01 was created on GitHub. In the same local repository I added a second remote called github and verified both remotes. Neither URL contains a token or password.

```
cd ~/Assignment01
git remote add github https://github.com/shanzaanum7-hub/Assignment01.git
git remote -v
```

```
shanzaanum7-hub@ubuntu:~/Assignment01$ git remote -v
gitea   http://shanzaanum7-hub@127.0.0.1:3000/shanzaanum7-hub/Assignment01.git (fetch)
gitea   http://shanzaanum7-hub@127.0.0.1:3000/shanzaanum7-hub/Assignment01.git (push)
github  https://github.com/shanzaanum7-hub/Assignment01.git (fetch)
github  https://github.com/shanzaanum7-hub/Assignment01.git (push)
shanzaanum7-hub@ubuntu:~/Assignment01$
```

Figure 4: Both gitea and github remotes (fetch and push) *(task2_remotes.png)*

Step 2.2: Push to GitHub

GitHub does not accept account passwords for Git over HTTPS, so I generated a classic personal access token with the repo scope and pasted it at the password prompt.

```
git push -u github main
```

```
shanzaanum7-hub@ubuntu:~/Assignment01$ git push -u github main
Username for 'https://github.com': shanzaanum7-hub
Password for 'https://shanzaanum7-hub@github.com':
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 2 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 275 bytes | 68.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/shanzaanum7-hub/Assignment01.git
 * [new branch]      main -> main
branch 'main' set up to track 'github/main'.
shanzaanum7-hub@ubuntu:~/Assignment01$
```

Figure 5: Successful push of main to the github remote *(task2_github_push.png)*

Step 2.3: Verify the GitHub repository

The repository https://github.com/shanzaanum7-hub/Assignment01 is public, owned by my account, and displays the rendered README with my name and registration number.

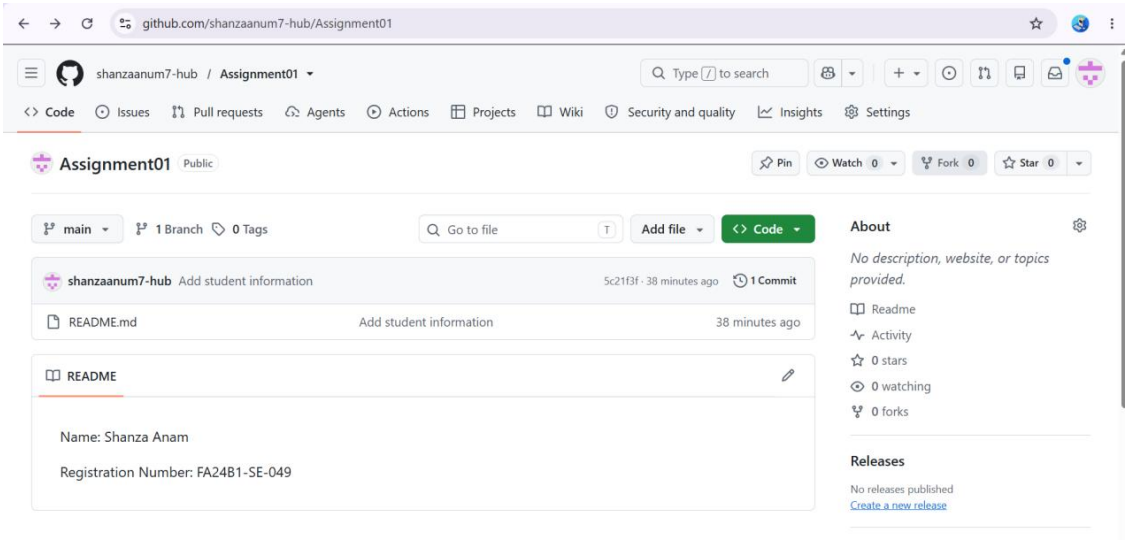


Figure 6: GitHub Assignment01 repository with rendered README *(task2_github_repository.png)*

Task 2 result: the same local repository is connected to both Gitea and GitHub and is publicly available on GitHub.

4. Task 3: Track Three Large Files with Git LFS

Step 3.1: Configure Git LFS

Git LFS 3.7.1 was installed from the Ubuntu repositories (sudo apt install -y git-lfs). I initialised it, confirmed the version, and configured tracking for all .bin files, which created the rule in .gitattributes.

```
cd ~/Assignment01
git lfs install
git lfs version
git lfs track "*.bin"
cat .gitattributes
```

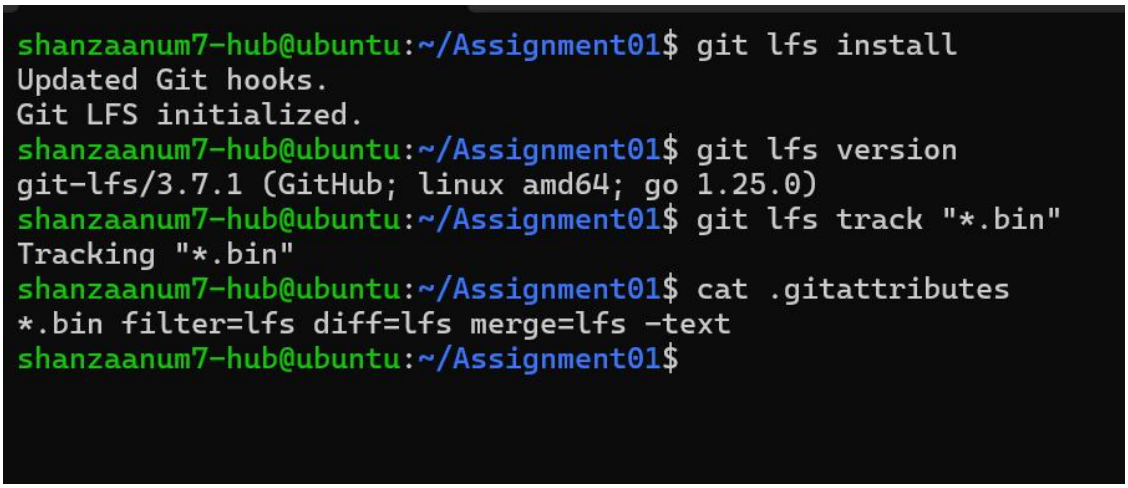


*Figure 7: Git LFS initialised, version shown, *.bin rule created* *(task3_lfs_setup.png)*

Step 3.2: Create and verify three LFS files

Three files larger than 100 MiB were created with truncate and staged together with .gitattributes. The command git lfs ls-files --size confirms all three are managed by LFS. Git LFS reports sizes in decimal megabytes, so 101, 102 and 103 MiB appear as 106, 107 and 108 MB; all are above 100 MiB.

```
truncate -s 101M large-file-1.bin
truncate -s 102M large-file-2.bin
truncate -s 103M large-file-3.bin
git add .gitattributes large-file-1.bin large-file-2.bin large-file-3.bin
git lfs ls-files --size
```

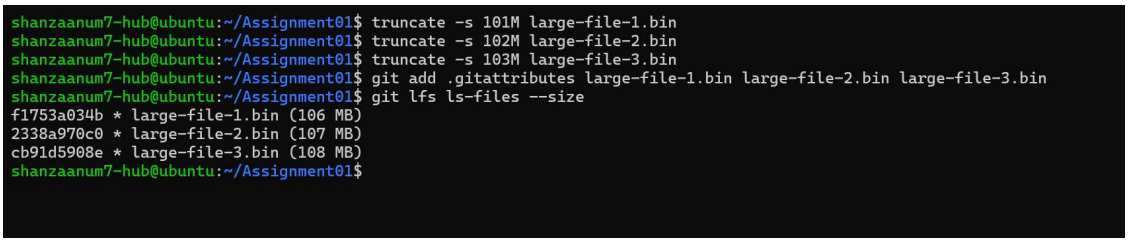


Figure 8: Three LFS-tracked files with sizes above 100 MiB *(task3_lfs_files.png)*

Step 3.3: Commit and push the LFS files

```
git commit -m "Add three files using Git LFS"
git push github main
```

Result: Uploading LFS objects: 100% (3/3), 321 MB completed successfully, followed by the normal push (5c21f3f..4b5b8ef main -> main).

```
shanzaanum7-hub@ubuntu:~/Assignment01$ git push github main
Username for 'https://github.com': shanzaanum7-hub
Password for 'https://shanzaanum7-hub@github.com':
Username for 'https://github.com': shanzaanum7-hub
Password for 'https://shanzaanum7-hub@github.com':
kUploading LFS objects:   0% (0/3), 174 MB | 945 KB/s
itUploading LFS objects:   0% (0/3), 174 MB | 944 KB/s
nUploading LFS objects:   0% (0/3), 175 MB | 944 KB/s
aUploading LFS objects:   0% (0/3), 175 MB | 944 KB/s
Uploading LFS objects: 100% (3/3), 321 MB | 925 KB/s, done.
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 2 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 796 bytes | 99.00 KiB/s, done.
Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/shanzaanum7-hub/Assignment01.git
   5c21f3f..4b5b8ef  main -> main
shanzaanum7-hub@ubuntu:~/Assignment01$
```

Figure 9: Git LFS upload completed 100% (3/3) and pushed to GitHub *(task3_lfs_push.png)*

Task 3 result: three files larger than 100 MiB are tracked with Git LFS and pushed to the GitHub Assignment01 repository.

5. Task 4: Portfolio with GitHub Pages

Step 4.1: Create and push the portfolio repository

A public repository named shanzaanum7-hub.github.io was created. The portfolio is built with plain HTML and CSS using two files: index.html and styles.css. The page contains the four required sections: About Me, Education, Skills and Projects. The files were committed and pushed to the main branch.

```
mkdir shanzaanum7-hub.github.io && cd shanzaanum7-hub.github.io
git init && git branch -M main
git add index.html styles.css
git commit -m "Add portfolio"
git remote add origin https://github.com/shanzaanum7-hub/shanzaanum7-hub.github.io.git
git push -u origin main
```

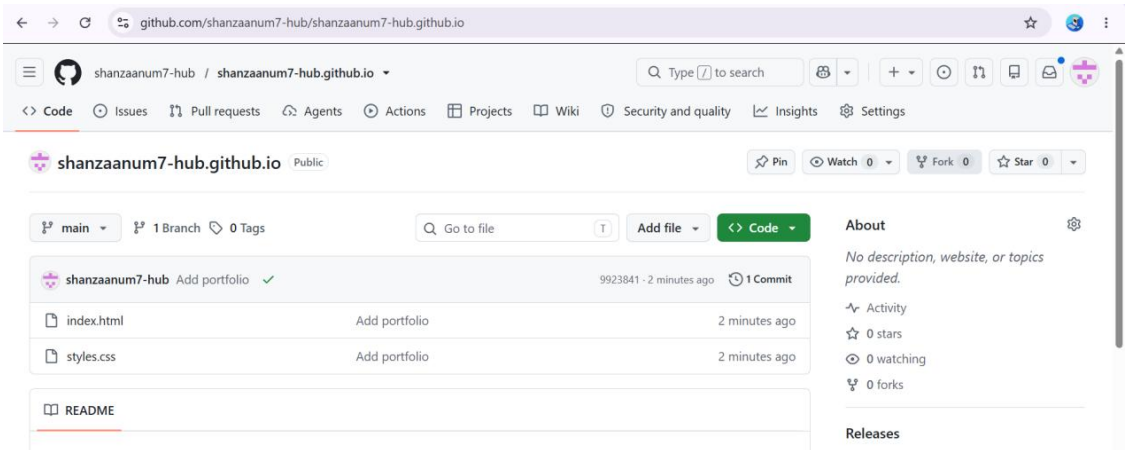


Figure 10: Portfolio repository containing index.html and styles.css *(task4_pages_repository.png)*

Step 4.2: Publish with GitHub Pages

In Settings, Pages, the source was set to Deploy from a branch with the main branch and the / (root) folder. GitHub reports that the site is live at https://shanzaanum7-hub.github.io/.

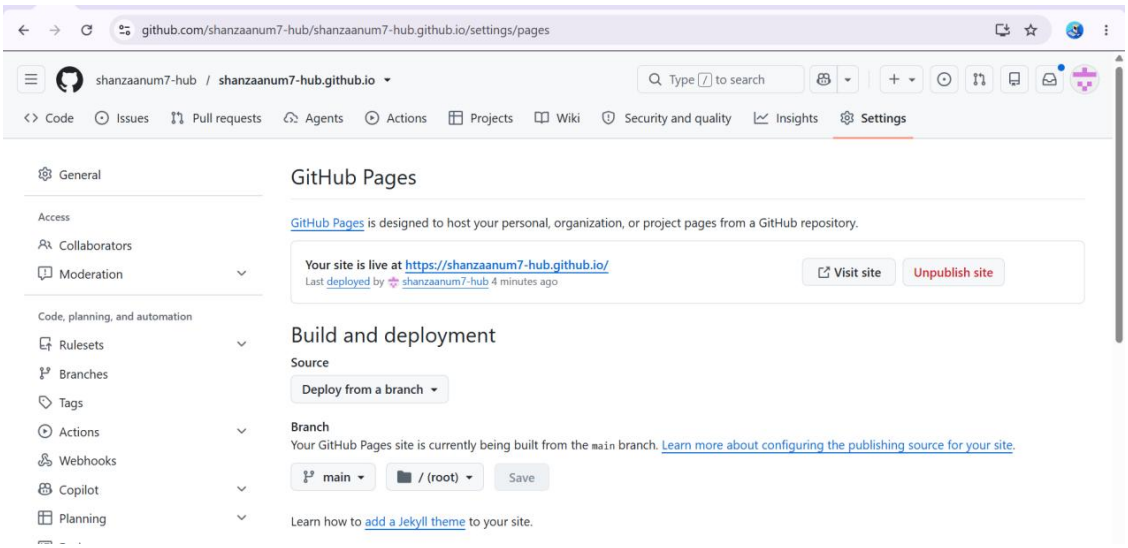


Figure 11: GitHub Pages settings showing the site is live *(task4_pages_deployment.png)*

Step 4.3: Verify the live portfolio

The published site loads successfully at https://shanzaanum7-hub.github.io/ and displays the portfolio content.

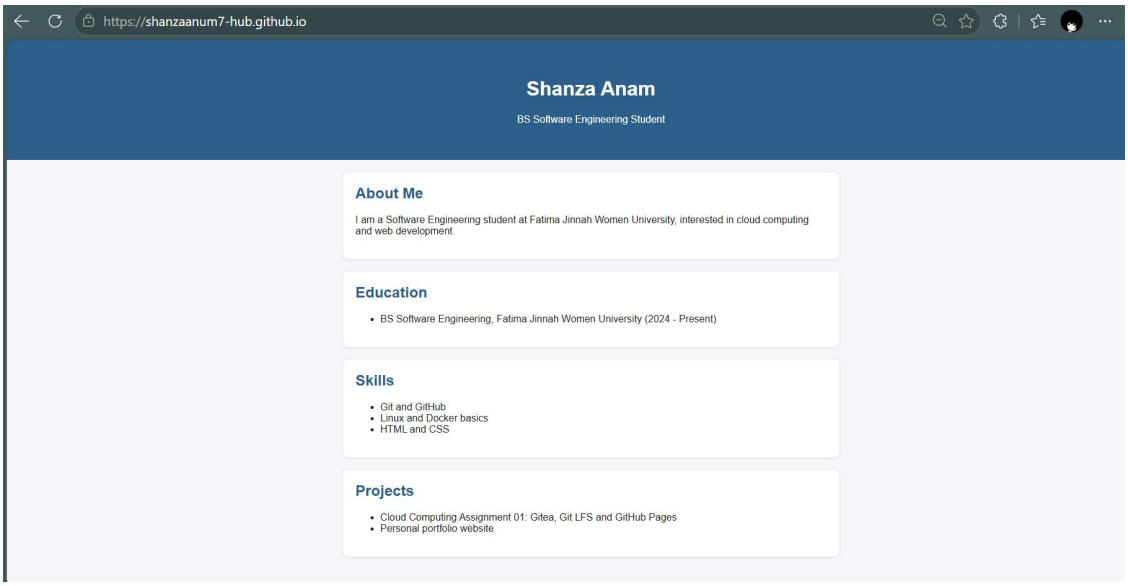


Figure 12: Live portfolio at shanzaanum7-hub.github.io *(task4_portfolio_live.png)*

Task 4 result: the portfolio is stored in a public GitHub repository and published as a live GitHub Pages site.

6. Summary of Deliverables

Repositories and links

Item	Link
Gitea repository	http://192.168.215.130:3000/shanzaanum7-hub/Assignment01
GitHub Assignment01	https://github.com/shanzaanum7-hub/Assignment01
Portfolio repository	https://github.com/shanzaanum7-hub/shanzaanum7-hub.github.io
Live portfolio	https://shanzaanum7-hub.github.io/

Screenshot index

No.	File name	Evidence
1	task1_gitea_running.png	Gitea and PostgreSQL running, HTTP 200
2	task1_gitea_push.png	Push to Gitea remote
3	task1_gitea_repository.png	Gitea repository with README
4	task2_remotes.png	Both remotes configured
5	task2_github_push.png	Push to GitHub
6	task2_github_repository.png	GitHub repository with README
7	task3_lfs_setup.png	Git LFS setup and .gitattributes
8	task3_lfs_files.png	Three LFS files above 100 MiB
9	task3_lfs_push.png	LFS upload 100% (3/3)
10	task4_pages_repository.png	Portfolio source files on GitHub
11	task4_pages_deployment.png	GitHub Pages deployment live
12	task4_portfolio_live.png	Live portfolio site

7. Conclusion

In this assignment I installed and ran a Gitea service using Docker on an Ubuntu server, created a repository on that server, and pushed it to two different remotes, Gitea and GitHub. I used personal access tokens entered only at password prompts, so no credential was stored in a remote URL, shell command or screenshot. I also tracked three files larger than 100 MiB with Git LFS and published a portfolio website with GitHub Pages. Together these tasks gave me hands-on experience with self-hosted and cloud-hosted Git services, container-based deployment, large file management and static site hosting.